FORMAN CHRISTIAN COLLEGE
(A CHARTERED UNIVERSITY)

COMP 451 COMPILER CONSTRUCTION B

SPRING 2025

Class Project

Compiler

Muhammad Zeeshan 261940660

M Zeeshan Mufti 251694851

Introduction:

We are going to focus on the code of a very basic compiler that has already been given to us in proj.c.

The code does a job of tokenizing a stream of code, then traversing through the tokens and determining the statement by those tokens if its an TOKEN_ID means its an assignment statement, if its TOKEN_TYPE means its type like int or float. If its TOKEN_SEMICOLON means it's the end of the current declaration or assignment statement:

Further on operations like this y=x+5 will be considered assignment statements in our code.

We have to implement Task 2 to 7 on proj.c and incrementally uodate our compiler

Task 1:

Following are the answers:

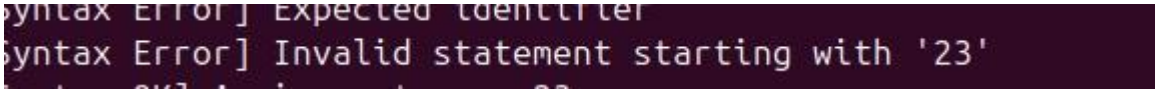
1- TokenType enum is nothing more than just to enumerate our TokenType that we have defined. Everything within enum is just like defining macros and allocating a number to each for everytime it is used in the code. The item at the very start would be 1 and then so on. If we added a new token in TokenType enum it would do nothing more than assign that type a number based on its position in the enum struct.

```
if (isalpha(input[i])) { // Identifier or type
    buffer_index = 0;
    while (isalpha(input[i])) {
        buffer[buffer_index++] = input[i++];
    }
    buffer[buffer_index] = '\0';
    i--;

    if (is_type(buffer)) {
        tokens[token_count].type = TOKEN_TYPE;
    } else {
        tokens[token_count].type = TOKEN_ID;
    }
```

2- We are incrementing I and accesing the ith index whenever we want to store the element of alphabet or number in the buffer. A stage will occur when we might skip over a space between the current token and the next one , so in order to avoid such a situation we move backward towards a space so that space can be skipped by our parse function.

3- The match function checks our current token that we are on and compares its type with the type that has been passed to it to confirm whether its that type or not. If it is then return 1. It helps in syntax analysis to tackle statements based on the first token they have. In parse we match with the type we want to make sure then tokens we are sending to a function actually work with that function (sending a token stream starting with id would be sent to declaration function). We need to increase current_token to move forward in the stream of tokens and continue on with the loop in parse().

4- The input like int 123 will give an error as we know declaration statement accepts TOKEN_TYPE followed by TOKEN_ID. Since id is handled by tokenize in a way that ids are only of type alphabet this kind of token is unexpected and is handled like:



5- We are storing declared variables in symbol table which is also a list of structs of Symbol type that have name and type as attributes. We are going through symbol at the time of assignment to make sure the ID we have is even part of the symbol table (declared). Found is a flag which is 0 till it becomes 1 when we find the variable in the symbol table. When its 1 then we continue with assignmnet like expected.

6- The parser has defined the syntax of all statement so if an invalid statement occurs it disturbs the expected flow of the program and of course it will follow an alternate path which simply displays an error. If such a situation occurs it will move on to the nearest semicolon and then roll back to parse function to focus on the next statement.

7- We could extend it to check type compatibility by comparing the token number with the type. The number is essentially a string so we can strchr to see if a . occurs in the number meaning its float so we match it with the type of the declared variable. If they are not the same then give an error or warning.

8- The skipping to the nearest semi colon works well. Based on the kinds of statements we are covering it is fine but if we were to cover functions, loops etc it would falter.

9- The symbol table stores the variables along with their type. Currently there is no consequences of having variables with the same name.

10- We could consider these statements as assignment statememts but extending and adding operator in between and cover the other twow operands as ids or number or a combination. A new token to recognize operators would need to be added.

Task 2:

In task 2 we are asked to add the functionality of accepting statements like x=y+5, which we will have to extend our assignment statement from accepting just a single number or variable to accepting three elements id/num operator id/num.

We chose assignment statement because it starts with a variable and has an equal sign already:

Following are the main changes and outputs

```
if (tokens[current_token].type == TOKEN_NUMBER &&
    tokens[current_token + 1].type == TOKEN_OPERATOR &&
    tokens[current_token + 2].type == TOKEN_ID &&
    tokens[current_token + 3].type == TOKEN_SEMICOLON) {

    char left[50], op[5], right[50];
    strcpy(left, tokens[current_token++].value);
    strcpy(op, tokens[current_token++].value);
    strcpy(right, tokens[current_token++].value);
    current_token++; // ;

    printf("[Syntax OK] Operation Statement: %s = %s %s %s;\n",
    return 1;
}
```

```
\n");
```

```
, left, op, right);
```

```
if (tokens[current_token].type == TOKEN_ID &&
    tokens[current_token + 1].type == TOKEN_OPERATOR &&
    tokens[current_token + 2].type == TOKEN_NUMBER &&
    tokens[current_token + 3].type == TOKEN_SEMICOLON) {

    char left[50], op[5], right[50];
    strcpy(left, tokens[current_token++].value);
    strcpy(op, tokens[current_token++].value);
    strcpy(right, tokens[current_token++].value);
    current_token++; // ;

    printf("[Syntax OK] Operation Statement: %s = %s %s %s;\n",
    return 1;
}
```

- After equals we have defined several cases for how the assignment statement can be like.
- By peeking the next few tokens we can easily make the accurate message and move current_token at the same time

For operator we have a new token defined which has two value + or – just like TOKEN_TYPE has int or float.

```
// Token types
typedef enum {
    TOKEN_TYPE,
    TOKEN_ID,
    TOKEN_NUMBER,
    TOKEN_EQUALS,
    TOKEN_SEMICOLON,
    TOKEN_OPERATOR,
    TOKEN_UNKNOWN,
} TokenType;
```

```
    }else if (input[i] == '+' || input[i] == '-') {
tokens[token_count].type = TOKEN_OPERATOR;
if (input[i]=='+'){
strcpy(tokens[token_count].value,"+");
token_count++;
}else{
strcpy(tokens[token_count].value,"-");
token_count++;
      }
```

Outputs:

```
Zeeshan@Ubuntu:~/Downloads$ gcc task2.c -o t2
Zeeshan@Ubuntu:~/Downloads$ ./t2
Input Code 1: int x; int y; x = 1 + 2; y = x - 5;float z;z=23;x=y-z;
[Syntax OK] Declaration: int x;
[Syntax OK] Declaration: int y;
[Syntax OK] Operation Statement: x = 1 + 2;
[Syntax OK] Operation Statement: y = x - 5;
[Syntax OK] Declaration: float z;
[Syntax OK] Assignment: z = 23;
[Syntax OK] Operation: x = y - z;

Input Code 2: int x; int y;float z;z=23;x=y-z;
[Syntax OK] Declaration: int x;
[Syntax OK] Declaration: int y;
[Syntax OK] Declaration: float z;
[Syntax OK] Assignment: z = 23;
[Syntax OK] Operation: x = y - z;

Input Code 4: int x; int y;y=100;x=22-y;
[Syntax OK] Declaration: int x;
[Syntax OK] Declaration: int y;
[Syntax OK] Assignment: y = 100;
[Syntax OK] Operation Statement: x = 22 - y;

Input Code 5: int x; int y;float z;int b;=y+z;a=b+c;y=x+z+b;
[Syntax OK] Declaration: int x;
[Syntax OK] Declaration: int y;
[Syntax OK] Declaration: float z;
[Syntax OK] Declaration: int b;
[Syntax Error] Invalid statement starting with '='
[Semantic Error] Variable 'a' not declared
[Syntax Error] Invalid assignment format after '='
```

Task 3:

In task 3 we are asked to add support for multiple variable declarations on one line.

This would be very because when a comma is detected in declaration statement we can assume a stream of variables and can loop through the tokens till semicolon

```
  else if (input[i]==','){
  tokens[token_count].type=TOKEN_SEPARATOR;
  strcpy(tokens[token_count].value,",");
token_count++;
  }
```

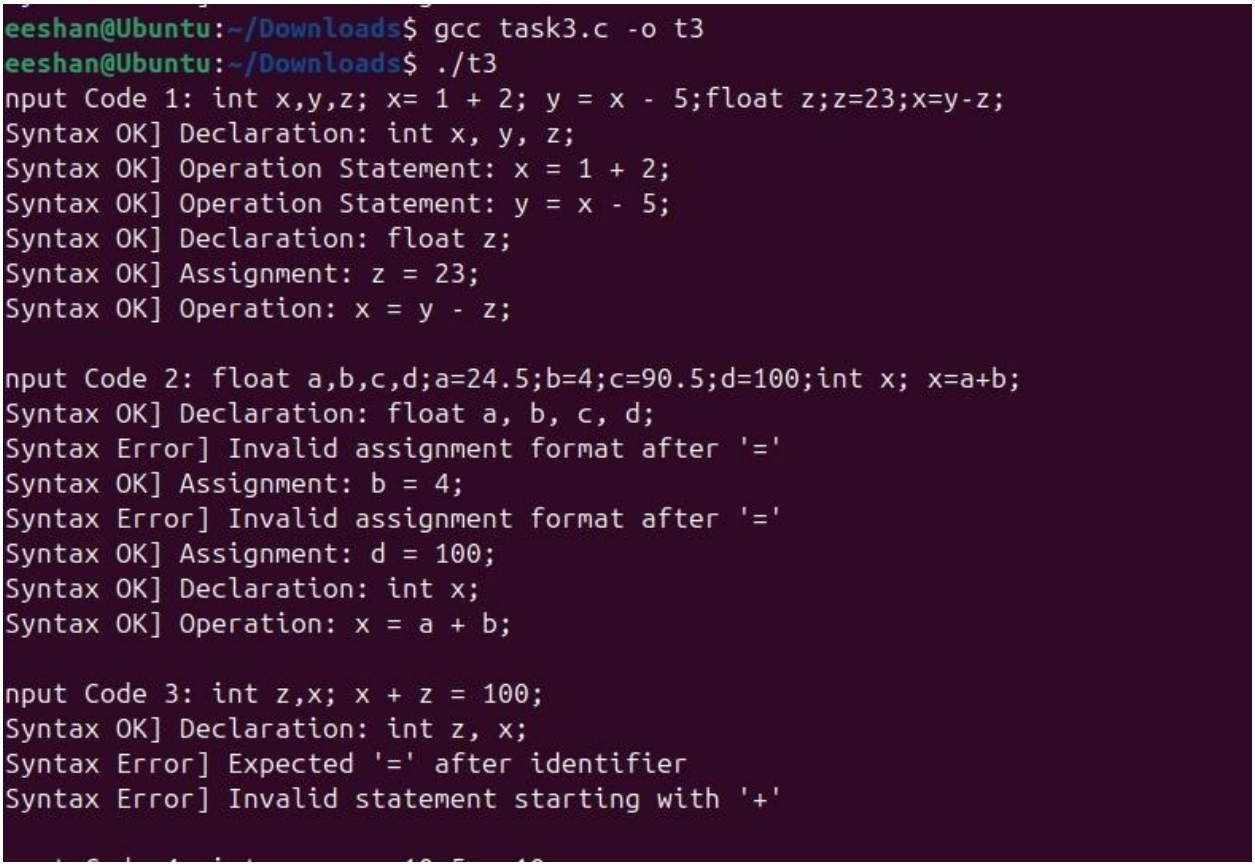
and store those tokens with their variable type in symbol table that will consider them officially declared

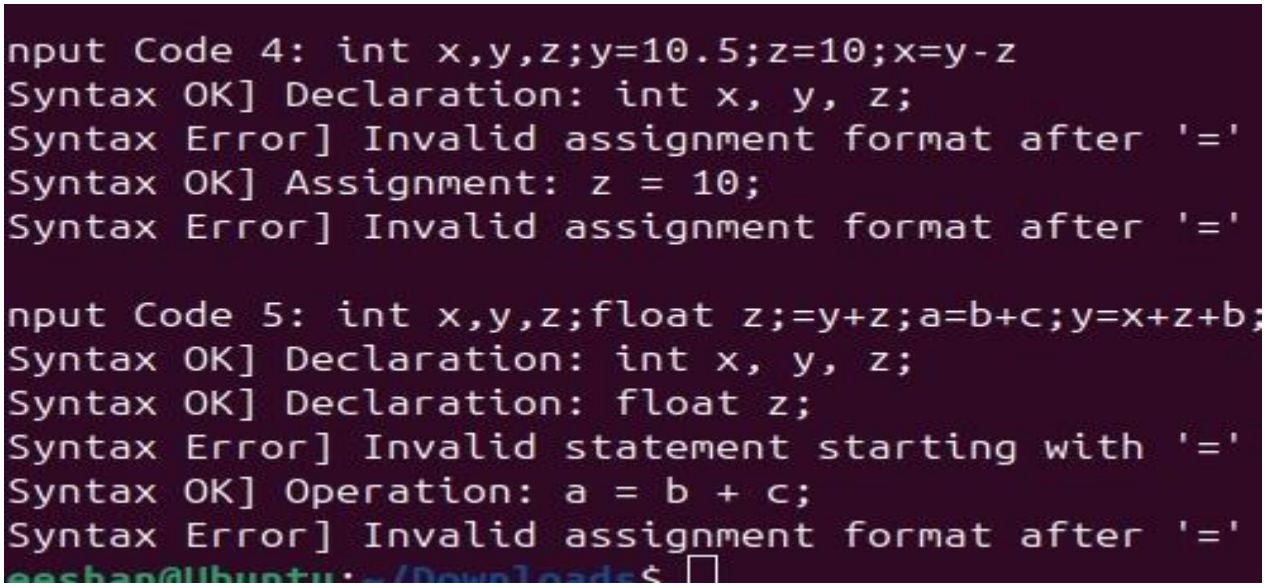
- To store variables to display them while giving the message.
- If comma detected, skip over it
- If an Id wasn't detected after comma it's a syntax error otherwise keep going on store the variable in symbol table and buffer. Move symbol count
- Keep doing till semicolon

```
if (tokens[current_token].type == TOKEN_SEPARATOR) {
    // Multiple variables declared
    while (tokens[current_token].type == TOKEN_SEPARATOR) {
        current_token++;  // consume comma

        if (!match(TOKEN_ID)) {
            printf("[Syntax Error] Expected identifier after ','\n");
            return 0;
        }

        strcpy(var_name, tokens[current_token - 1].value);
        strcpy(symbol_table[symbol_count].name, var_name);
        strcpy(symbol_table[symbol_count].type, var_type);
        strcpy(buffer[i++], var_name);
        symbol_count++;
    }

    if (match(TOKEN_SEMICOLON)) {
        printf("[Syntax OK] Declaration: %s ", var_type);
        for (int j = 0; j < i; j++) {
            printf("%s", buffer[j]);
            if (j < i - 1) printf(", ");
        }
        printf(";\n");
        return 1;
```

- Once semicolon is detected
- Print the message in the format and move on.

Outputs:





Task 4:

In task 4 we are asked to implement type checking in assignmnets and declarations furthermore declarations like int x=10; are supposed to be accepted where both assignment and declaration is being done:

```
if (tokens[current_token].type == TOKEN_ID &&
    tokens[current_token + 1].type == TOKEN_SEMICOLON) {
    char val[50];
    strcpy(val, tokens[current_token++].value);
    current_token++; // ;

    char right_type[10];
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbol_table[i].name, val) == 0)
            strcpy(right_type, symbol_table[i].type);
    }

    if (strcmp(var_type, "int") == 0 && strcmp(right_type, "float") == 0) {
        printf("[Truncation] Unmatched datatype with '%s'\n", var_name);
    }else{

    printf("[Syntax OK] Assignment: %s = %s;\n", var_name, val);
    return 1;
    }
}

printf("[Syntax Error] Invalid assignment format after '=' or unmatched datatype\n");
return 0;
```

```
Multiple variables
ile (tokens[current_token].type == TOKEN_SEPARATOR) {
  current_token++;
  strcpy(var_name, tokens[current_token - 1].value);
  strcpy(buffer[i++], var_name);
  if (!match(TOKEN_ID)) {
      printf("[Syntax Error] Expected identifier after ','\n");
      return 0;
  }

  strcpy(var_name, tokens[current_token - 1].value);
  strcpy(buffer[i++], var_name);

  //Assigning value to the multiple variables
  if (tokens[current_token].type == TOKEN_EQUALS) {
      current_token++;
      if (tokens[current_token].type == TOKEN_NUMBER) {
          strcpy(init_val, tokens[current_token].value);
          if (strcmp(var_type, "int") == 0 && strchr(init_val, '.')) {
              printf("[Truncation] Cannot assign float to int variable '%s'\n", var_name);
              return 0;
          }else if (strcmp(var_type, "float") == 0 && !strchr(init_val, '.')) {
              printf("[Truncation] Cannot assign int to float variable '%s'\n", var_name);
              return 0;
          }
```

- Take this for example we check its type before orinting the confirmed operation statement. If the types of the variable don’t match by checking the symbol table it should be a warning.
- This shows how we have implemented it in declarations first storing all variables insymbol table then when = is detected we check if the next element is number.
- Initval is buffer in which we store the number
- Check our variables type since all variabke are stored in one line so they are the same so we can check which ever was recently in var_type and match it with the number if number has . In it it is float.
- Give appropriate warning messages

```
Input Code 4: float x,y,z=12.3; int z=12; x = y + 10;y=x+z;
[Syntax OK] Declaration: float x,y,z
[Syntax OK] Declaration: int z
[Syntax OK] Operation Statement: x = y + 10;
[Truncation] Unmatched datatype with 'y'
[Syntax Error] Invalid assignment format after '=' or unmatc

Input Code 5: float a, b, c; a = 2.2; b = 3.3; c = a + b;
[Syntax OK] Declaration: float a,b,c
[Syntax OK] Assignment: a = 2.2;
[Syntax OK] Assignment: b = 3.3;
[Syntax OK] Operation Statement: c = a + b;
```

```
Zeeshan@Ubuntu:~/Downloads$ ./t4
Input Code 1: int x, y; float z; x = 5 + 2.3; y = 4; z = x + y;
[Syntax OK] Declaration: int x,y
[Syntax OK] Declaration: float z
[Truncation] Unmatched datatype with 'x'
[Syntax Error] Invalid assignment format after '=' or unmatched datatype
[Syntax OK] Assignment: y = 4;
[Truncation] Unmatched datatype with 'z'
[Syntax Error] Invalid assignment format after '=' or unmatched datatype

Input Code 2: int a; a = 12.7;
[Syntax OK] Declaration: int a
[Truncation] Unmatched datatype with 'a'
[Syntax Error] Invalid assignment format after '=' or unmatched datatype

Input Code 3: float f=10; int x=10.5; f = 4.2; x = f;
[Truncation] Cannot assign int to float variable 'f'
[Syntax Error] Invalid statement starting with '10'
[Truncation] Cannot assign float to int variable 'x'
[Syntax Error] Invalid statement starting with '10.5'
[Syntax OK] Assignment: f = 4.2;
[Truncation] Unmatched datatype with 'x'
[Syntax Error] Invalid assignment format after '=' or unmatched datatype
```

Outputs:

Task 5:

In task 5 we are asked to support duplicate variable declaration we can do this by having an array where we store all variables then traversing that array everytime we declare a variable.

This means intx;int x; or int x; float x; will give error and x will only have datatype of the first declaration statement in the assignment statements furthermore int a,b,a will also give an error as we will keep comparing with our variables array

We also have a dupdetected function which print a message returns 1 on any string given ti it we only give a string to it when we have detected duplicate ourselves by going through variables which is list containg our variables

```
// Handle additional comma-separated variables
while (tokens[current_token].type == TOKEN_SEPARATOR) {
    current_token++; // consume ','

    if (!match(TOKEN_ID)) {
        printf("[Syntax Error] Expected identifier after ','\
        goto end_declaration;
    }

    strcpy(var_name, tokens[current_token - 1].value);

    // Check duplicate
    for (int d = 0; d < var_count; d++) {
        if (strcmp(variables[d], var_name) == 0) {
            if (dupdetected(var_name)) goto end_declaration;
        }
    }
    strcpy(variables[var_count++], var_name);
    strcpy(symbol_table[symbol_count].name, var_name);
    strcpy(symbol_table[symbol_count].type, var_type);
    symbol_count++;
    strcpy(buffer[i++], var_name);
```

end_declaration is a label that just moves to the next semicolon.

```
int dupdetected(const char *str){
    printf("[Semantic Error] Variable %s already declared\n",str);
    return 1;
    }
```

Outputs:

```
Zeeshan@Ubuntu:~/Downloads$ ./t5
Input Code 1: int x, y; float x; x = 5 + 2.3; y = 4; z = x + y;
[Syntax OK] Declaration: int x, y;
[Semantic Error] Variable x already declared
[Truncation] Unmatched datatype with 'x'
[Syntax Error] Invalid assignment format after '=' or unmatched datatype
[Syntax OK] Assignment: y = 4;
[Semantic Error] Variable 'z' not declared

Input Code 2: int x;int x;
[Syntax OK] Declaration: int x;
[Semantic Error] Variable x already declared

Input Code 3: float f=10; int x=10.5; f = 4.2; x = f;
[Truncation] Cannot assign int to float variable 'f'
[Truncation] Cannot assign float to int variable 'x'
[Syntax OK] Assignment: f = 4.2;
[Truncation] Unmatched datatype with 'x'
[Syntax Error] Invalid assignment format after '=' or unmatched datatype

Input Code 4: float x,x,x=12.3; int z=12; x = y + 10;y=x+z;
[Semantic Error] Variable x already declared
[Syntax OK] Declaration: int z;
[Syntax OK] Operation Statement: x = y + 10;
[Semantic Error] Variable 'y' not declared

Input Code 5: float a, b, a; a = 2.2; b = 3.3; c = a + b;
[Semantic Error] Variable a already declared
[Syntax OK] Assignment: a = 2.2;
[Syntax OK] Assignment: b = 3.3;
[Semantic Error] Variable 'c' not declared
```

Task 6:

In task 6, we are asked to provide support for code described within {}.

I decided that it would make sense to create a new token type that supports this. When this is detected in parse we turn on a flag variable and adjust our declaration and assignment statement to print a message saying in scope to allow us to pick up that there is scope support in the code.

```
while (current_token < token_count) {
    if (tokens[current_token].type == TOKEN_SCOPE) {
        if (tokens[current_token].value[0] == '{') {
            inscope = 1;
            pvar = 0;
        } else {
            inscope = 0;
            pvar = 0;
            for (int i = 0; i < 100; i++) privar[i][0] = '\0';
        }
        current_token++;
        continue;
    }
```

In parse()

```
char variables[100][50];
int var_count = 0;

char privar[100][50];
int pvar = 0;

int inscope = 0;
```

```
// Token types
typedef enum {
    TOKEN_TYPE,
    TOKEN_ID,
    TOKEN_NUMBER,
    TOKEN_EQUALS,
    TOKEN_SEMICOLON,
    TOKEN_OPERATOR,
    TOKEN_SEPARATOR,
    TOKEN_SCOPE,
    TOKEN_UNKNOWN
} TokenType;
```

Globally defined privar to separate variables defined in scope from global variables. A privsymbol table is also created to support type checking when carrying out operations in scope.

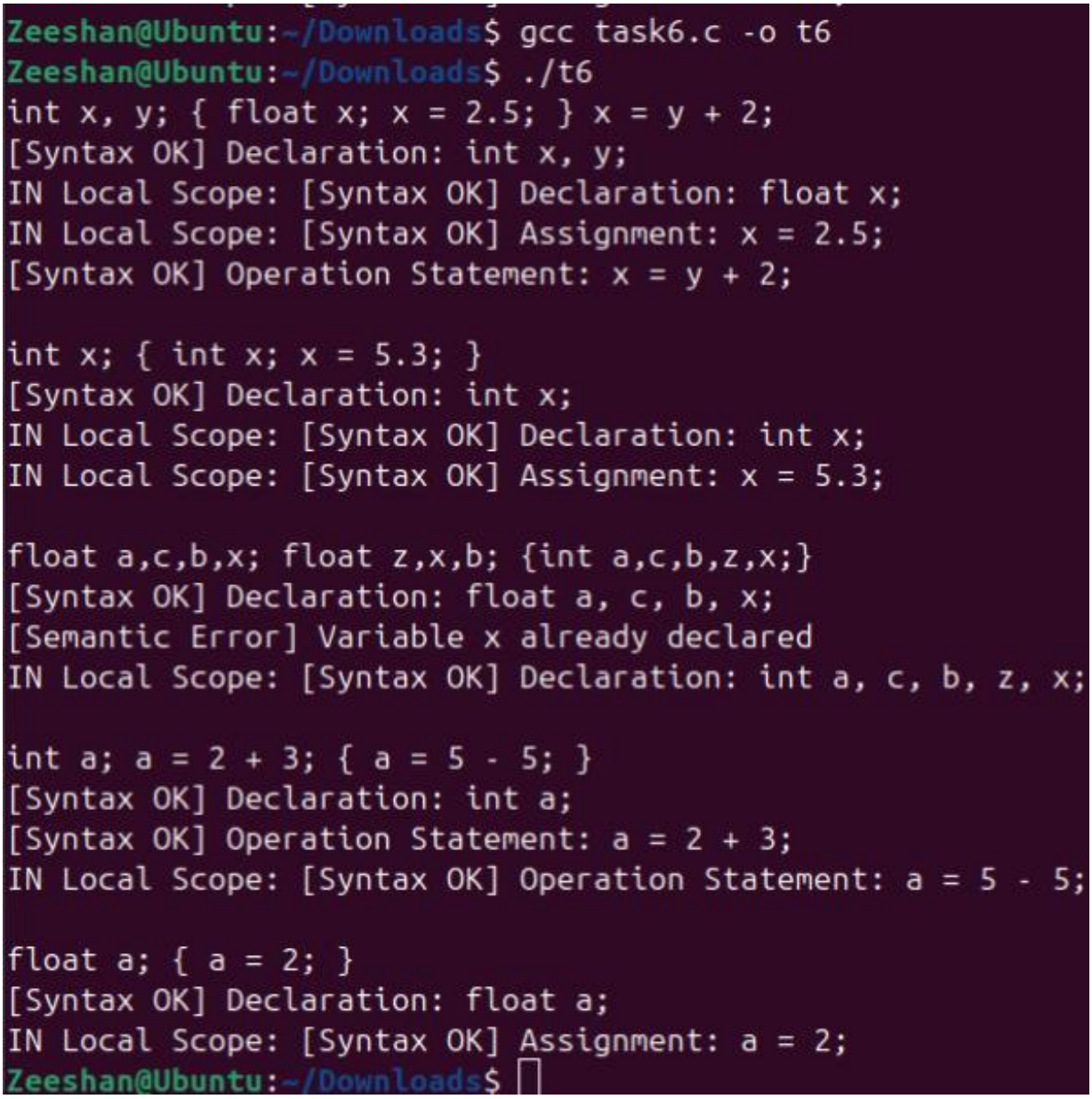
```
if (inscope) printf("IN Local Scope: ");
printf("[Truncation] Cannot assign %s to %s variable '%s'\n",
       strchr(init_val, '.') ? "float" : "int", var_type, var_name);
goto end_decl;
```

Messages are written like this while in scope

```
int found = 0;
char var_type[10];

if (inscope) {
    for (int i = 0; i < privsym; i++) {
        if (strcmp(privsymbol[i].name, var_name) == 0) {
            found = 1;
            strcpy(var_type, privsymbol[i].type);
            break;
        }
    }
    if (!found) {
        printf("IN Local Scope: ");
        printf("[Semantic Error] Variable '%s' not declared\n", var_name);
        while (current_token < token_count && tokens[current_token].type != TOKEN_SEMICOLON)
            current_token++;
        if (current_token < token_count) current_token++;
        return 0;
    }
} else {
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbol_table[i].name, var_name) == 0) {
            found = 1;
            strcpy(var_type, symbol_table[i].type);
            break;
        }
    }
    if (!found) {
        printf("[Semantic Error] Variable '%s' not declared\n", var_name);
        while (current_token < token_count && tokens[current_token].type != TOKEN_SEMICOLON)
            current_token++;
        if (current_token < token_count) current_token++;
        return 0;
    }
}
```

If in scope we check private symbol table, if a variable is declared globally with a but not in scope. A statement like a=2 would give a not declared

Outputs:



Task 7:

Task 7 is very simple we just have to write line numbers next to error messages specifically.

After every resolution statement we will increment line number because we are sure a line is done.

Line number is cleared before another code is tokenized or during tokenization.

```
printf("[Truncation] Cannot assign %s to %s variable '%s' Line:%d\n",
      strchr(init_val, '.') ? "float" : "int", var_type, var_name,lineno);
      lineno++;
```

```
int lineno=0;
```

Outputs:

```
Input Code 1: int x, y; float x; x = 5 + 2.3; y = 4; z = x + y;
[Syntax OK] Declaration: int x, y;
[Semantic Error] Variable x already declared Line:1
[Truncation] Unmatched datatype with 'x' Line:1
[Syntax Error] Invalid assignment format after '=' or unmatched datatype Line:2
[Syntax OK] Assignment: y = 4;
[Semantic Error] Variable 'z' not declared Line: 4

Input Code 2: int x;int x;
[Syntax OK] Declaration: int x;
[Semantic Error] Variable x already declared Line:1

Input Code 3: float f=10; int x=10.5; f = 4.2; x = f;
[Truncation] Cannot assign int to float variable 'f' Line:0
[Truncation] Cannot assign float to int variable 'x' Line:1
[Syntax OK] Assignment: f = 4.2;
[Truncation] Unmatched datatype with 'x' Line:3
[Syntax Error] Invalid assignment format after '=' or unmatched datatype Line:4

Input Code 4: float x,x,x=12.3; int z=12; x = y + 10;y=x+z;
[Semantic Error] Variable x already declared Line:0
[Syntax OK] Declaration: int z;
[Syntax OK] Operation Statement: x = y + 10;
[Semantic Error] Variable 'y' not declared Line: 1

Input Code 5: float a, b, a; a = 2.2; b = 3.3; c = a + b;
[Semantic Error] Variable a already declared Line:0
[Syntax OK] Assignment: a = 2.2;
[Syntax OK] Assignment: b = 3.3;
[Semantic Error] Variable 'c' not declared Line: 2
Zeeshan@Ubuntu:~/Downloads$
```